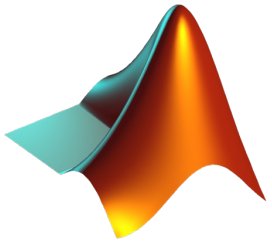


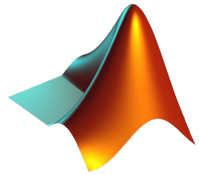
MATLAB® Tutorial: Basic Usage

Data Science and Decision Making (DSDM)

Indian Institute of Technology Gandhinagar, Gujarat

Pratyush Dayal

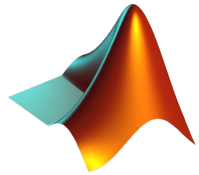
Variables, Operators, and Precedence



- **Variables**
  - Entities that store a certain data under a given name
- **Operators**
  - Numeric
  - Relational
  - Special character
- **Operator precedence**
  - Different operators have different hierarchy
  - Operations of same precedence are carried out from left to right

( )	Parentheses
' .' ^ .^	Transpose Power
^+ ^-	Power with unary plus or minus
+ - ~	Unary plus, unary minus, negation
* / \ .* ./ .\	Matrix multiplication, division, elementwise multiplication, division
+ -	Addition, Subtraction
:	Colon operator
< <= > >= == ~=	Relational operators
&	Elementwise AND
|	Elementwise OR

Operator precedence - Examples



❑ **Example 1:**

```
(5*(80/(4+24/(1+1)*3))-1)^0.5
=(5*(80/(4+24/(1+1)*3))-1)^0.5
=(5*(80/(4+24/2*3))-1)^0.5
=(5*(80/(4+24/2*3))-1)^0.5
=(5*(80/(4+12*3))-1)^0.5
=(5*(80/(4+36))-1)^0.5
=(5*(80/40)-1)^0.5
=(5*(80/40)-1)^0.5
=(5*2-1)^0.5
=(10-1)^0.5
=(10-1)^0.5
=9^0.5
= 3
```

MATLAB® output

```
>> (5 * (80 /(4 + 24 / (1 + 1) * 3)) - 1)^0.5

ans =

     3
```

Algebraic equation to MATLAB® expressions

❑ **Example 2:**

$$\left(\frac{a}{b+(cd-a^2)}\right)^{\left(\frac{b}{c}\right)}$$

Solution:

```
1.  ()^()
2.  (()/())^(b/c)
3.  ((a)/(()+()))^(b/c)
4.  ((a)/((b)+()))^(b/c)
5.  ( (a) / ( (b) + (c*d - a^2) ) )^(b/c)
6.  ( a / ( b + (c*d - a^2) ) )^(b/c)
```

Arrays and Matrices

- **1D Array/Vector**
  - A collection of values in either a row or a column form
  - Elements of an array are separated by a space or a comma

```
>> a = [1, 2, 3, 4, 5]
a =
     1     2     3     4     5
>> a = [1 2 3 4 5]
a =
     1     2     3     4     5
```

Defining a 1D array

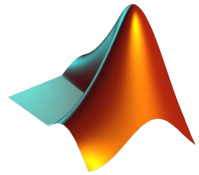
- **2D Array/Matrix**
  - A matrix is a collection of data, organised into rows and columns
  - Each row is separated by a semicolon ;

```
>> A = [1 2 3; 4 5 6]

A =

     1     2     3
     4     5     6

>> A = [1,2,3;4,5,6]

A =

     1     2     3
     4     5     6
```

Defining a 2D array

- **Multidimensional array**
  - Simply an array of arrays

Creating 1D arrays in multiple ways



- **Let A, B and C be a row-vectors of order $1 \times m$.**
- **Multiple ways to initialise a 1D array:**

```
>> A = [4 8 9 -3 5.5 9 11 13.74 -3];
>> A = [4, 8, 9, -3, 5.5, 9, 11, 13.74, -3];
```

- **Using colon operator**
  - Arrays that follow an arithmetic progression can be initialised using the colon (:) operator

```
>> B = 5:10

B =

     5     6     7     8     9    10
```

  - Evenly spaced arrays can be created as follows

```
>> B = 5:2:20

B =

     5     7     9    11    13    15    17    19
```

  - The arrays could be created in a decreasing trend too. For example,

```
>> C = 37:-4:8

C =

    37    33    29    25    21    17    13     9
```

  - Using `linspace()` function

```
>> A = linspace(1,10,5)
A =
    1.0000    3.2500    5.5000    7.7500   10.0000
```

Some basic operations with arrays

- Let A and B be a row-vectors of order $1 \times m$
- Let C be a column vector of order $n \times 1$
- To access i^{th} element of a 1D array, follow the syntax `arrayname(i)`

```
>> A = [4 8 9 -3 5.5 9 11 13.74 -3];
>> A(5)
ans =
    5.5000
```

```
>> C = [4; 5;-3; 2.5]
C =
    4.0000
    5.0000
   -3.0000
    2.5000
>> C(2)
ans =
     5
```

The colon (:) operator

- The : operator is used to subscript arrays

```
>> A = [4 8 9 -3 5.5 9 11 13.74 -3];
>> A(3:7)
ans =
    9.0000   -3.0000    5.5000    9.0000   11.0000
```

- If the last index is unknown, `end` keyword is used

```
>> A(5:end)
ans =
    5.5000    9.0000   11.0000   13.7400   -3.0000
```

- Since a vector is considered as a matrix in MATLAB®, the following is valid too

```
>> A(1,4:6)
ans =
   -3.0000    5.5000    9.0000
```

Creating 2D matrices in multiple ways

- **Let A, B and C be a row-vectors of order $m \times n$**
- **Let C be a matrix of order $p \times q$**
- **Multiple ways to initialise a matrix:**

```
>> A = [4, 5, 8; -2, 6, 9; 1, 7, 11]

A =

     4     5     8
    -2     6     9
     1     7    11
```

```
>> A = [[4, 5, 8]; [-2, 6, 9]; [1, 7, 11]]
A =
     4     5     8
    -2     6     9
     1     7    11
```

- Matrix can be created by concatenating multiple arrays of **same size**

```
>> a = [6 8 3 4];
>> b = [2 -1 0 9];
>> C = [a; b]
C =
     6     8     3     4
     2    -1     0     9
```

- A bigger matrix can be created by concatenating smaller matrices

```
>> a = [1 0; 3 -5];
>> b = [2 7; 8 9];
>> C = [a, b]

C =

     1     0     2     7
     3    -5     8     9
```

Some basic operations with matrices

- Let A and B be a matrices of order $m \times n$
- Let C be a matrix of order $p \times q$
- Conjugate transpose of a matrix

```
>> A
A =
     3     5    -9     6
    -1     2     5     9
     7    -2    -3     4
>> A'
ans =
     3    -1     7
     5     2    -2
    -9     5    -3
     6     9     4
```

The colon (:) operator

- The : operator is used to subscript matrices

```
>> B = A(1:3, 2:4)
B =
     5    -9     6
     2     5     9
    -2    -3     4
```

```
>> C = A(1:3, 2:end)
C =
     5    -9     6
     2     5     9
    -2    -3     4
>> D = A(:, 2:end)
D =
     5    -9     6
     2     5     9
    -2    -3     4
```

Matrix operations

- **Let A and C be a matrices of order $m \times n$**
- **Let B be a matrix of order $p \times q$**
- **Matrix multiplication**
  - Matrices A and B can be multiplied iff $n = p$

```
>> A = [[3 4 -8];[2 1 0]; [7 -3 0]];
>> B = [[1 0 -4 7];[2 2 -6 3];[9 -5 5 8]];
>> A*B
ans =
   -61    48   -76   -31
     4     2   -14    17
     1    -6   -10    40
```

```
>> B = B';
>> A*B
Error using  *
Incorrect dimensions for matrix multiplication. Check that the
number of columns in the first matrix matches the number of rows in
the second matrix. To operate on each element of the matrix
individually, use TIMES (.*) for elementwise multiplication.
```

- **The . operator is used to specify elementwise operation**
  - Elementwise operations can be performed on matrices of same dimensions only

```
>> A = [[3 4 -8];[2 1 0]; [7 -3 0]];
>> B = [[0 -4 7];[2 2 -6];[9 -5 8]];
>> C = A.*B
C =
     0   -16   -56
     4     2     0
    63    15     0
```

```
>> C = A./B
C =
       Inf   -1.0000   -1.1429
    1.0000    0.5000         0
    0.7778    0.6000         0
```

Further matrix manipulations and operations

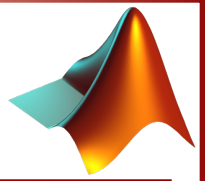
- **Let A and B be a matrices of order $m \times n$**
- **Let C be a matrix of order $p \times q$**
- **Reshaping a matrix**
  - `reshape(A,[m,n])` reshapes the matrix `A` into a matrix of size $m \times n$ columnwise
  - Gives an error if the matrix `A` does not contain mn elements

```
>> A = [[3 4 1 -8];[2 3 1 0]; [9 -3 0 5]]
A =
     3     4     1    -8
     2     3     1     0
     9    -3     0     5
>> reshape(A,[2,6])
ans =
     3     9     3     1     0     0
     2     4    -3     1    -8     5
```

- **Repeating a matrix using `repmat()`**
  - `repmat(a,[m,n])` creates a larger matrix where `m` rows and `n` columns of the matrix `a` are present

```
>> a = [1 2 3; 4 5 6];
>> A = repmat(a,[4,2])
A =
     1     2     3     1     2     3
     4     5     6     4     5     6
     1     2     3     1     2     3
     4     5     6     4     5     6
     1     2     3     1     2     3
     4     5     6     4     5     6
     1     2     3     1     2     3
     4     5     6     4     5     6
```

Inbuilt special matrices



❑ Matrix with all entries as 1

```
>> ones(3,4)
ans =
     1     1     1     1
     1     1     1     1
     1     1     1     1
```

❑ Null matrix

```
>> zeros(2,5)
ans =
     0     0     0     0     0
     0     0     0     0     0
```

❑ Magic matrix

```
>> magic(3)
ans =
     8     1     6
     3     5     7
     4     9     2
```

❑ Diagonal matrix

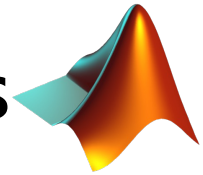
```
>> a = [-1, 4, 7];
>> diag(a)
ans =
    -1     0     0
     0     4     0
     0     0     7
```

```
>> A = [3, 5, -9, 6; -1, 2, 5, 9; 7 -2 -3 4];
>> B = diag(A)
B =
     3
     2
    -3
```

❑ Identity matrix

```
>> eye(3)
ans =
     1     0     0
     0     1     0
     0     0     1
```

Some useful inbuilt commands/keywords



`clc`
Clears the command window

`clear` *variablename*
Clears the variable '*variablename*'

`clf`
Clears the contents of a figure

`format` *style*
Sets the output display style

`figure`
Opens a new figure window

`close` *figurename*
Closes the figure '*figurename*'

`print`
Prints the current figure

`save`
Saves the current variables in the workspace

Mathematical functions & Trigonometrical functions

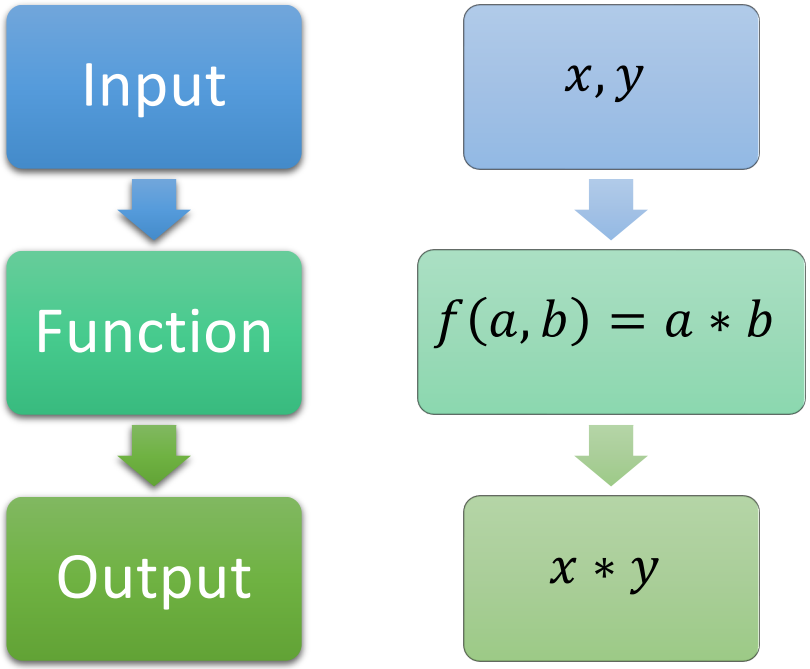
`sin(x)`	Sine
`cos(x)`	Cosine
`tan(x)`	Tangent
`asin(x)`	Arc sine
`acos(x)`	Arc cosine
`atan(x)`	Arc tangent
`log(x)`	Natural logarithm
`log10(x)`	Logarithm base-10
`exp(x)`	Exponential
`sqrt(x)`	Square root

`abs(x)`	Absolute value
`sgn(x)`	Signum function
`max(x)`	Maximum value
`min(x)`	Minimum value
`ceil(x)`	Round towards $+\infty$
`floor(x)`	Round towards $-\infty$
`round(x)`	Round to nearest integer
`rem(x,y)`	Remainder after division
`inv(A)`	Inverse of matrix A
`conj(x)`	Complex conjugate

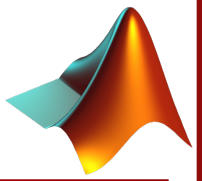
`pi`	π
`i j`	Imaginary root ($\sqrt{-1}$)
`Inf`	∞
`NaN`	Not a Number

MATLAB® Functions

- **Functions**
  - A function is a reusable block of code that performs a specific task or computation
  - Variables defined in a function are local to that function
  - A function performs the specific task only when called
- **Why use functions?**
  - To avoid duplicate blocks of code
  - Separate complex operations
  - Make error identification and debugging easier
  - Promote program reuse

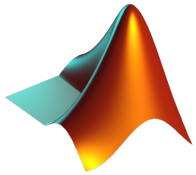


Inbuilt functions



`det`	Determinant of a square matrix (numerically and symbolically)
`diag`	Diagonal of a square matrix; creates a diagonal matrix
`dot`	Dot product of two vectors
`eig`	Eigenvalues and eigenvectors of special matrix equations
`end`	Last index in an array
`eye`	Creates the identity matrix
`find`	Finds indices of a vector satisfying a logical expression
`fliplr`	Flips elements of an array from left to right
`flipud`	Flips elements of an array from bottom to top
`inv`	Inverse of a square matrix (numerically and symbolically)
`length`	Length of a vector
`magic`	Creates a square matrix whose sum of each row, each column, and diagonals is equal
`max`	Determines the maximum value in an array
`min`	Determines the minimum value in an array
`ones`	Creates an array whose elements equal 1
`plot`	Plots curves in a plane using linear axes
`rank`	Estimates number of linearly independent rows or columns of a matrix
`repmat`	Replicates arrays
`size`	Order (size) of an array
`sort`	Sorts elements of an array in ascending order
`sum`	Sums elements of an array
`zeros`	Creates an array whose elements equal 0

User-Defined Functions



❑ **Syntax of a user-defined function:**

`function` [*outputvariables*] = ***FunctionName*** (*inputvariables*)
Expressions
outputvariables = results; % Assign the results to outputvariables
`end`

- **`function`:** Keyword to specify a function file
- *outputvariables*: One or more variables that will be returned by the function
- ***FunctionName***: This is the name given to the function. In MATLAB®, a function file is saved under the same name as that of the function
- *inputvariables*: The (local) variables which the function will use for its computation

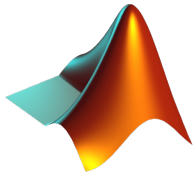
Example

```
function area = calcArea(L, W)
    % Area of a rectangle
    area = L * W;
end
```

To use this function:

```
Lnt = 5;
Wdt = 10;
rectArea = calcArea(Lnt,Wdt);
```

Example of a user defined function



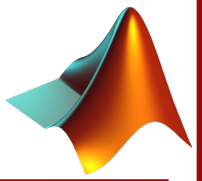
- Write a function that computes the functions x and y where

$$x = \cos(at) + b$$
$$y = |x| + c$$

- Solution:
  - Here, a, b, and c are scalars
  - t, x, and y are vectors
  - The function must accept t, a, b, c as input and return x, y as output
  - Let us name the function ComputeXY

```
function [x, y] = ComputeXY(t, a, b, c)
        % ComputeXY accepts t as a vector, and a, b, c as scalars
        % ComputeXY returns x, y as row vectors
        x  = cos(a*t) + b;
        y = abs(x) + c;
end
```

Alternate ways



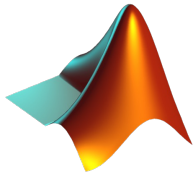
Function	Accessing function from script or function	Comments

`function z = ComputeXY(t, w)`
`x = cos(w(1)*t)+w(2);`
`z = [x; abs(x)+w(3)];`
	`t = 0:pi/4:pi;`
	`w = [1.4, 2, 0.75];`
	`q = ComputeXY(t, w);`
		$w(1) = a = 1.4$; $w(2) = b = 2$;
		$w(3) = c = 0.75$; $q \rightarrow (2 \times 5)$
		$x(:) = q(1, 1:5)$; $y(:) = q(2, 1:5)$

`function z = ComputeXY(t, w)`
`x = cos(w(1)*t)+w(2);`
`z = [x abs(x)+w(3)];`
	`t = 0:pi/4:pi;`
	`w = [1.4, 2, 0.75];`
	`q = ComputeXY(t, w);`
		$w(1) = a = 1.4$; $w(2) = b = 2$;
		$w(3) = c = 0.75$; $q \rightarrow (1 \times 10)$
		$x(:) = q(1:5)$; $y(:) = q(6:10)$

`function[x, y] = ComputeXY(t, w)`
`x = cos(w(1)*t)+w(2);`
`y = abs(x)+w(3);`
	`t = 0:pi/4:pi;`
	`w = [1.4, 2, 0.75];`
	`q = ComputeXY(t, w);`
		$w(1) = a = 1.4$; $w(2) = b = 2$;
		$w(3) = c = 0.75$; $q \rightarrow (1 \times 5)$
		$x = q$; y not available[§]

Nested Functions



- **Syntax of nested functions:**

```
function [outputvariables] = FunctionName1 (inputvariables)
   Expressions
   function [outputvariables] = FunctionName2(inputvariables)
   …
   end
   function [outputvariables] = FunctionNamen(inputvariables)
   …
   end
end
```

- Further, multiple functions can be written in a single function file
- Filename must be the same as that of the first function

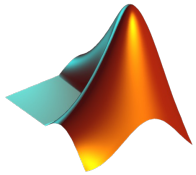
Example

```
function volume = calcVol(h, r)
    % Volume of a cylinder
    function area = calcArea(r)
        area = pi*r*r;
    end
    baseArea = calcArea(r);
    volume = baseArea * h;
end
```

To use this function:

```
rad = 5;
height = 10;
volCyl = calcVol(height,rad);
```

Anonymous functions



- **Anonymous functions**
  - These are means to create functions for simple expressions without having to create function files
  - Can be created using @ symbol in the command window, script file, primary function or a subfunction
  - Syntax: **functionhandle** = @(arguments) (expression)
  - Here,
    - **functionhandle** is the handle of the function, i.e., a variable that stores the association to a function
    - arguments is a comma-separated list of variable names
    - expression is a valid MATLAB® expression

$$c_x = |\cos(\beta x)|$$

```
cx = @(x, bet) (abs(cos(bet*x)));
disp(cx(4.1, pi/3))
```